Vitale_Ana500

October 26, 2025

1 ANA 500 Week 1: Ddata Organization & Analysis

1.0.1 Vincent Vitale

1.0.2 Instructor: Professor Ghaznavy

1.0.3 Date: October 5, 2025

1.0.4 Problem Statement

Airline companies aim to enhance customer satisfaction and reduce delays. Using passenger and operational flight data, this project will identify which flight-related and service-related factors most influence overall passenger satisfaction and on-time performance.

H (Satisfaction): Flight- and service-related variables have no effect on passenger satisfaction.

H : At least one flight or service variable (e.g., class, type of travel, seat comfort) has a significant effect on satisfaction.

H (On-time performance): Operational characteristics in this dataset (e.g., flight distance) have no effect on departure delay.

H : At least one operational characteristic (e.g., distance groups) shows a systematic difference in average departure delay.

[35]:
```
# Import packages and data
import time
import matplotlib
import pandas as pd
import numpy as np, joblib
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.widgets import Slider, RadioButtons, CheckButtons

# imports for modeling
from sklearn.base import clone
from sklearn.model_selection import train_test_split, cross_val_score,␣
 ↪GridSearchCV, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, r2_score,␣
 ↪root_mean_squared_error
```

```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.svm import SVR
from sklearn.dummy import DummyRegressor
from IPython.display import Image, display
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.neural_network import MLPRegressor


# Airline data:
AIRLINE_DATA = r"C:\Users\vince\Desktop\School\ANA500\airline.csv"
```

2 Organize Data

[6]:
```
# View data first to ensure its loaded correctly:
airlines = pd.read_csv(AIRLINE_DATA)

airlines.head()
```

[6]:

	Unnamed: 0	id	Gender	Customer Type	Age	Type of Travel	\
0	0	70172	Male	Loyal Customer	13	Personal Travel	
1	1	5047	Male	disloyal Customer	25	Business travel	
2	2	110028	Female	Loyal Customer	26	Business travel	
3	3	24026	Female	Loyal Customer	25	Business travel	
4	4	119299	Male	Loyal Customer	61	Business travel	

	Class	Flight Distance	Inflight wifi service	\
0	Eco Plus	460	3	
1	Business	235	3	
2	Business	1142	2	
3	Business	562	2	
4	Business	214	3	

	Departure/Arrival time convenient	…	Inflight entertainment	\
0	4	…	5	
1	2	…	1	
2	2	…	5	
3	5	…	2	
4	3	…	3	

	On-board service	Leg room service	Baggage handling	Checkin service	\
0	4	3	4	4	
1	1	5	3	1	
2	4	3	4	4	
3	2	5	3	1	

```
4                 3                 4                 4                 3

   Inflight service  Cleanliness  Departure Delay in Minutes  \
0                 5            5                          25
1                 4            1                           1
2                 4            5                           0
3                 4            2                          11
4                 3            3                           0

   Arrival Delay in Minutes             satisfaction
0                      18.0  neutral or dissatisfied
1                       6.0  neutral or dissatisfied
2                       0.0                satisfied
3                       9.0  neutral or dissatisfied
4                       0.0                satisfied

[5 rows x 25 columns]
```

[7]:
```
# Inspect the structure of the data
print("Shape:", airlines.shape)
airlines.info()
```

```
Shape: (129880, 25)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 25 columns):
 #   Column                            Non-Null Count   Dtype
---  ------                            --------------   -----
 0   Unnamed: 0                        129880 non-null  int64
 1   id                                129880 non-null  int64
 2   Gender                            129880 non-null  object
 3   Customer Type                     129880 non-null  object
 4   Age                               129880 non-null  int64
 5   Type of Travel                    129880 non-null  object
 6   Class                             129880 non-null  object
 7   Flight Distance                   129880 non-null  int64
 8   Inflight wifi service             129880 non-null  int64
 9   Departure/Arrival time convenient 129880 non-null  int64
 10  Ease of Online booking            129880 non-null  int64
 11  Gate location                     129880 non-null  int64
 12  Food and drink                    129880 non-null  int64
 13  Online boarding                   129880 non-null  int64
 14  Seat comfort                      129880 non-null  int64
 15  Inflight entertainment            129880 non-null  int64
 16  On-board service                  129880 non-null  int64
 17  Leg room service                  129880 non-null  int64
 18  Baggage handling                  129880 non-null  int64
 19  Checkin service                   129880 non-null  int64
```

```
 20  Inflight service                  129880 non-null  int64
 21  Cleanliness                       129880 non-null  int64
 22  Departure Delay in Minutes        129880 non-null  int64
 23  Arrival Delay in Minutes          129487 non-null  float64
 24  satisfaction                      129880 non-null  object
dtypes: float64(1), int64(19), object(5)
memory usage: 24.8+ MB
```

[8]:
```
# First thing I want to do is change all the string value types to either␣
 ↪binary or categorical
# Gender: Male = 1, Female = 0
airlines['Gender'] = airlines['Gender'].map({'Male': 1, 'Female': 0})

# Customer Type: Loyal = 1, Disloyal = 0
airlines['Customer Type'] = airlines['Customer Type'].map({
    'Loyal Customer': 1,
    'disloyal Customer': 0
})

# Type of Travel: Business = 1, Personal = 0
airlines['Type of Travel'] = airlines['Type of Travel'].map({
    'Business travel': 1,
    'Personal Travel': 0
})

# Class: Business = 2, Eco Plus = 1, Eco = 0 (or adjust as you prefer)
airlines['Class'] = airlines['Class'].map({
    'Business': 2,
    'Eco Plus': 1,
    'Eco': 0
})

# Satisfaction: satisfied = 1, neutral or dissatisfied = 0
airlines['satisfaction'] = airlines['satisfaction'].map({
    'satisfied': 1,
    'neutral or dissatisfied': 0
})

distance_col = 'Flight Distance_Adjusted' if 'Flight Distance_Adjusted' in␣
 ↪airlines.columns else 'Flight Distance'

# Validate the changes:
airlines[['Gender', 'Customer Type', 'Type of Travel', 'Class',␣
 ↪'satisfaction']].head()
```

[8]:

	Gender	Customer Type	Type of Travel	Class	satisfaction
0	1	1	0	1	0

```
1       1               0               1       2               0
2       0               1               1       2               1
3       0               1               1       2               0
4       1               1               1       2               1
```

[9]:
```
# Values look good, lets check for missing data now
airlines.isnull().sum()
```

[9]:
```
Unnamed: 0                            0
id                                    0
Gender                                0
Customer Type                         0
Age                                   0
Type of Travel                        0
Class                                 0
Flight Distance                       0
Inflight wifi service                 0
Departure/Arrival time convenient     0
Ease of Online booking                0
Gate location                         0
Food and drink                        0
Online boarding                       0
Seat comfort                          0
Inflight entertainment                0
On-board service                      0
Leg room service                      0
Baggage handling                      0
Checkin service                       0
Inflight service                      0
Cleanliness                           0
Departure Delay in Minutes            0
Arrival Delay in Minutes            393
satisfaction                          0
dtype: int64
```

[10]:
```
# Since only 1 column has null values, need to find out why its null and if we need this data or not.
# Compare missing vs non-missing delay values
missing_arrival = airlines[airlines['Arrival Delay in Minutes'].isnull()]
non_missing_arrival = airlines[airlines['Arrival Delay in Minutes'].notnull()]

print("Rows with missing Arrival Delay:", len(missing_arrival))
print("Rows without missing Arrival Delay:", len(non_missing_arrival))

# Average departure delay in both groups
print("Avg Departure Delay (missing):", missing_arrival['Departure Delay in Minutes'].mean())
```

```
print("Avg Departure Delay (non-missing):", non_missing_arrival['Departure␣
 ↪Delay in Minutes'].mean())
```

```
Rows with missing Arrival Delay: 393
Rows without missing Arrival Delay: 129487
Avg Departure Delay (missing): 37.88549618320611
Avg Departure Delay (non-missing): 14.643385050236704
```

[11]:
```
# Should check if there is a relationship between the missing values, and on␣
 ↪time flights
airlines[airlines['Arrival Delay in Minutes'].isnull()][['Departure Delay in␣
 ↪Minutes']].value_counts()
```

[11]:
```
Departure Delay in Minutes
0                           147
4                            11
1                            11
2                            10
16                            6
                            ...
116                           1
118                           1
119                           1
121                           1
530                           1
Name: count, Length: 121, dtype: int64
```

[12]:
```
# Visualize the missing data too
airlines[['Departure Delay in Minutes', 'Arrival Delay in Minutes']].describe()
```

[12]:

	Departure Delay in Minutes	Arrival Delay in Minutes
count	129880.000000	129487.000000
mean	14.713713	15.091129
std	38.071126	38.465650
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	12.000000	13.000000
max	1592.000000	1584.000000

[13]:
```
# Since the missing data is a small amount of the data, and its very closely␣
 ↪related to on time departures im making
# the call to input the missing data with 0 values
airlines['Arrival Delay in Minutes'] = airlines['Arrival Delay in Minutes'].
 ↪fillna(0)

# Verify all null values have been replaced
airlines['Arrival Delay in Minutes'].isnull().sum()
```

[13]: 0

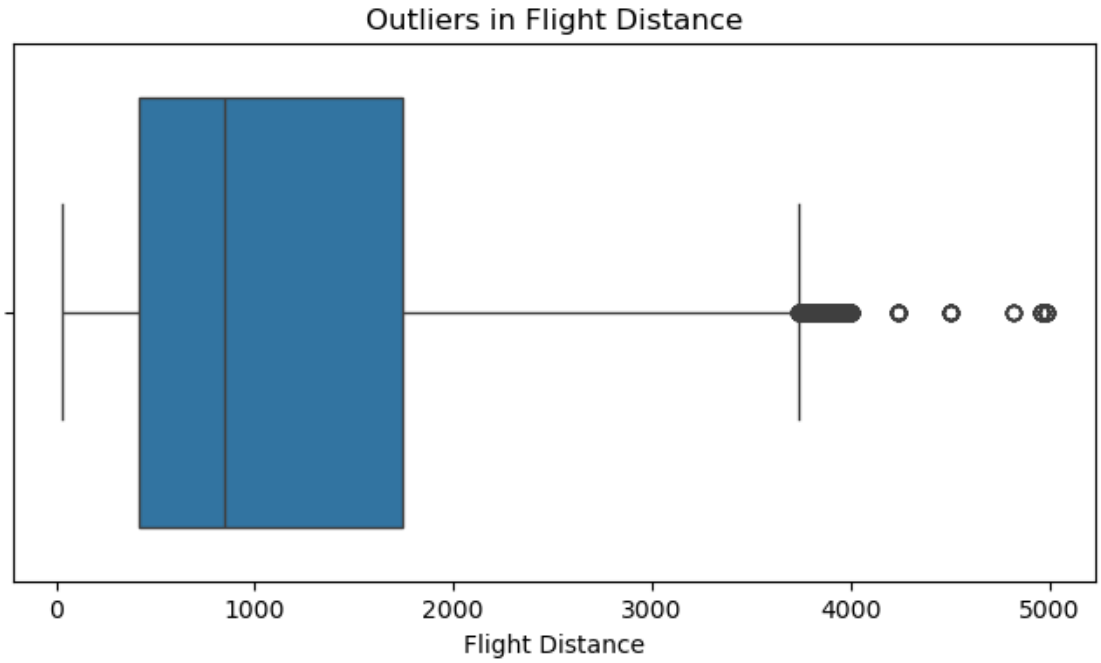
2.0.1 Detecting Outliers on Flight Distance

[14]:
```
q1, q3 = airlines["Flight Distance"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5*iqr, q3 + 1.5*iqr
airlines['Flight Distance_Capped'] = airlines["Flight Distance"].
 ↪clip(lower=low, upper=high)

airlines[["Flight Distance", "Flight Distance_Capped"]].describe()
```

[14]:

	Flight Distance	Flight Distance_Capped
count	129880.000000	129880.000000
mean	1190.316392	1186.995681
std	997.452477	988.394696
min	31.000000	31.000000
25%	414.000000	414.000000
50%	844.000000	844.000000
75%	1744.000000	1744.000000
max	4983.000000	3739.000000

[15]:
```
plt.figure(figsize=(8,4))
sns.boxplot(x=airlines["Flight Distance"])
plt.title("Outliers in Flight Distance")
plt.show()
```

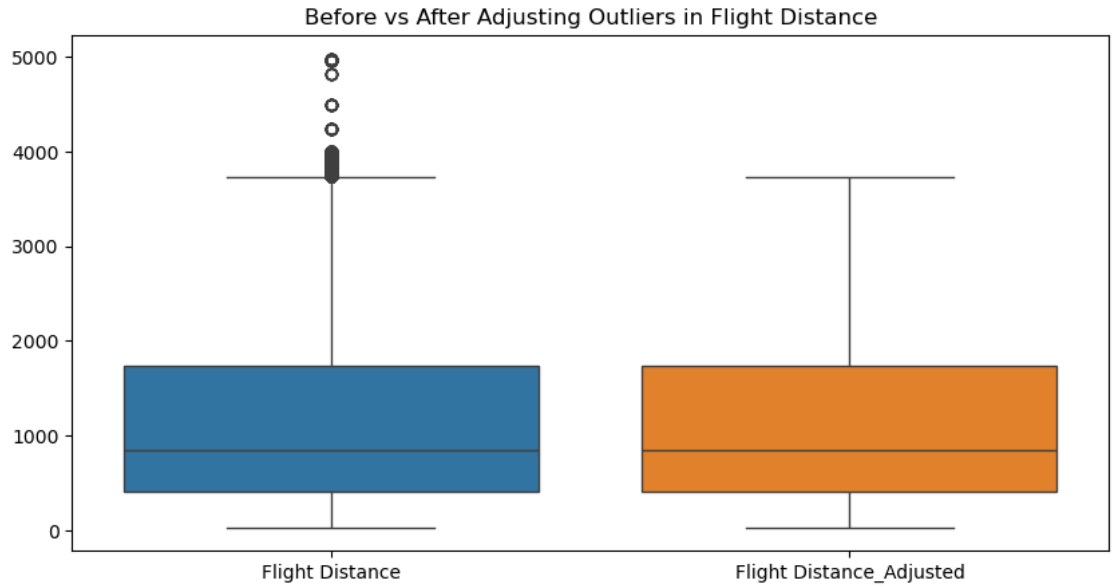


[16]:
```
# I do have some outlier data, I have opted to not remove the data and instead just reassign it to the highest value of q3 of the iqr.
airlines["Flight Distance_Adjusted"] = np.where(
    airlines["Flight Distance"] > high,
    high,
    airlines["Flight Distance"]
)

airlines[["Flight Distance", "Flight Distance_Adjusted"]].describe()
```

[16]:

	Flight Distance	Flight Distance_Adjusted
count	129880.000000	129880.000000
mean	1190.316392	1186.995681
std	997.452477	988.394696
min	31.000000	31.000000
25%	414.000000	414.000000
50%	844.000000	844.000000
75%	1744.000000	1744.000000
max	4983.000000	3739.000000

[17]:
```
plt.figure(figsize=(10,5))
sns.boxplot(data=airlines[["Flight Distance", "Flight Distance_Adjusted"]])
plt.title("Before vs After Adjusting Outliers in Flight Distance")
plt.show()
```



3 Analyzing Data

3.0.1 Questions to answer now that our data is cleaned and organized:

3.0.2 1. Average Satisfaction by Class

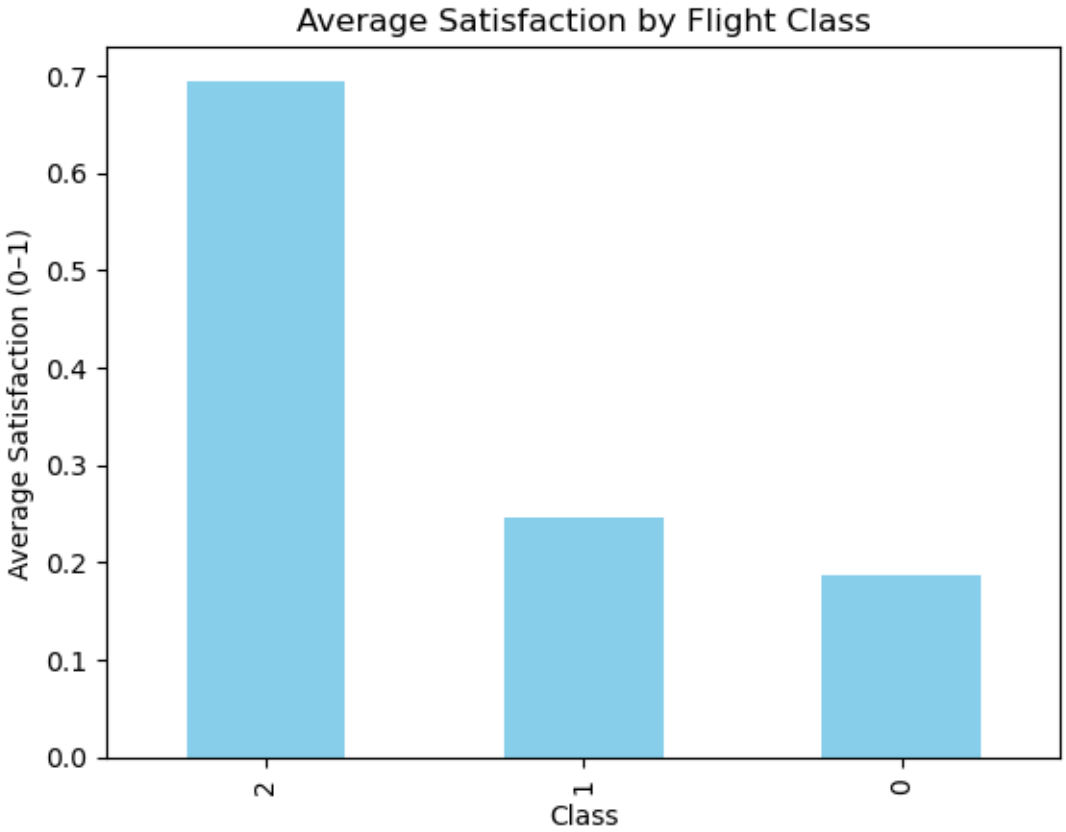
3.0.3 2. Average Flight Distance by Type of Travel

3.0.4 3. Satisfaction by Customer Type

3.0.5 4. Pivot Table (Class vs Type of Travel)

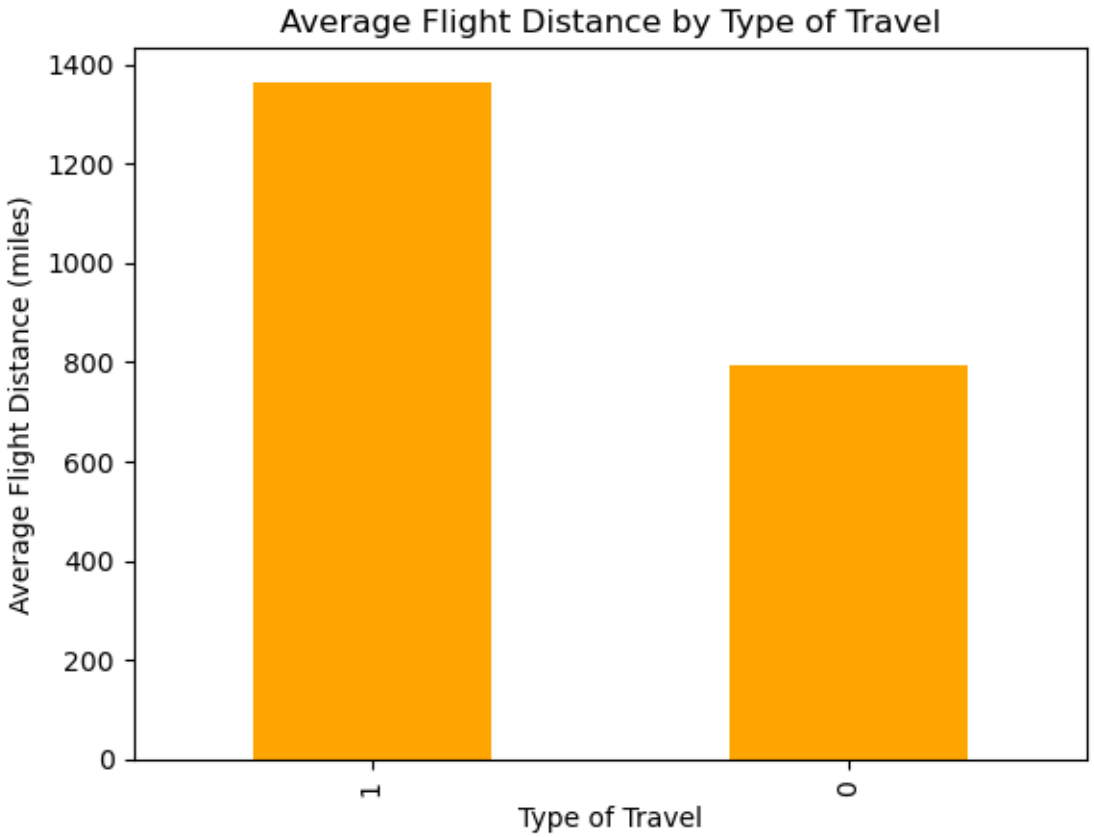
3.0.6 5. Aggregate Average Departure Delay by Distance Range

[18]:
```
# Average satisfaction score by class
satisfaction_by_class = airlines.groupby("Class")["satisfaction"].mean().
 ↪sort_values(ascending=False)
satisfaction_by_class

satisfaction_by_class.plot(kind="bar", title="Average Satisfaction by Flight␣
 ↪Class", color="skyblue")
plt.xlabel("Class")
plt.ylabel("Average Satisfaction (0-1)")
plt.show()
```



Avg Satisfaction by Class: Business class shows the highest average satisfaction, indicating service tier is a strong driver of perceived quality.

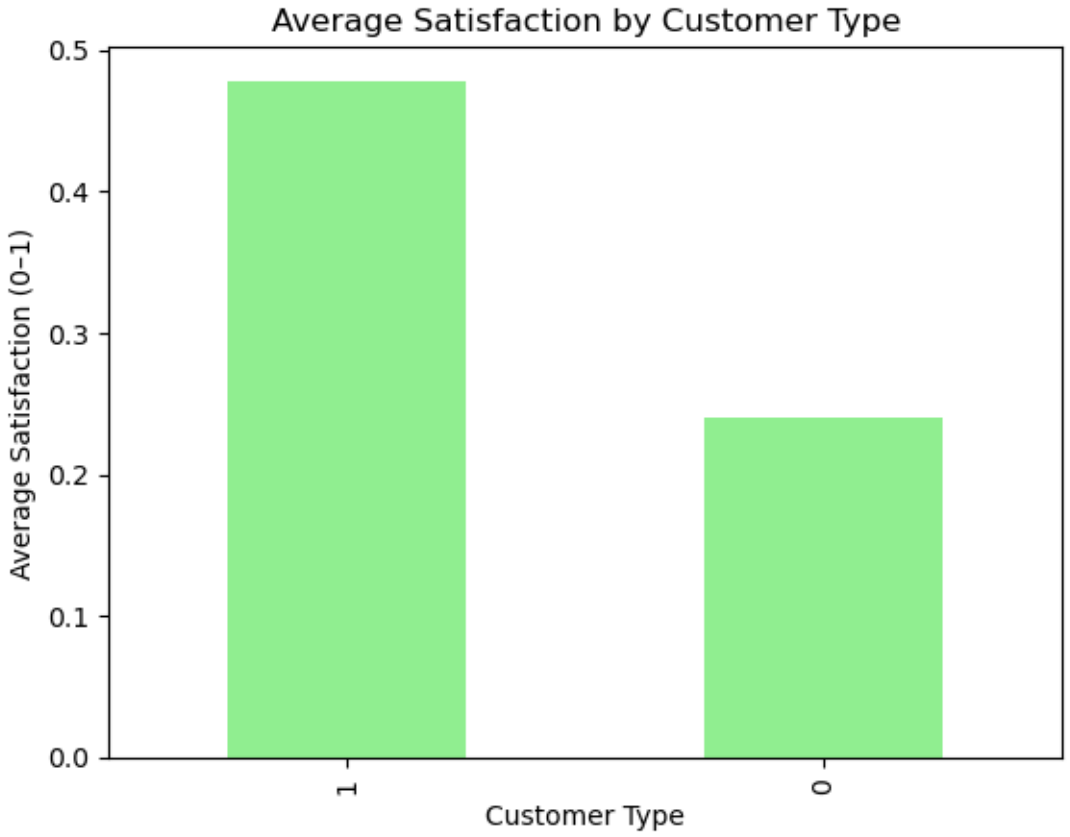
[19]:
```
# Average Flight Distance by Type of Travel
distance_by_travel = airlines.groupby("Type of Travel")["Flight␣
 ↪Distance_Adjusted"].mean().sort_values(ascending=False)
distance_by_travel

distance_by_travel.plot(kind="bar", title="Average Flight Distance by Type of␣
 ↪Travel", color="orange")
plt.xlabel("Type of Travel")
plt.ylabel("Average Flight Distance (miles)")
plt.show()
```



Avg Distance by Type of Travel: Business travel tends to cover longer routes, which may correlate with different expectations and satisfaction profiles.

[20]:
```
# Satisfaction by Customer Type
```

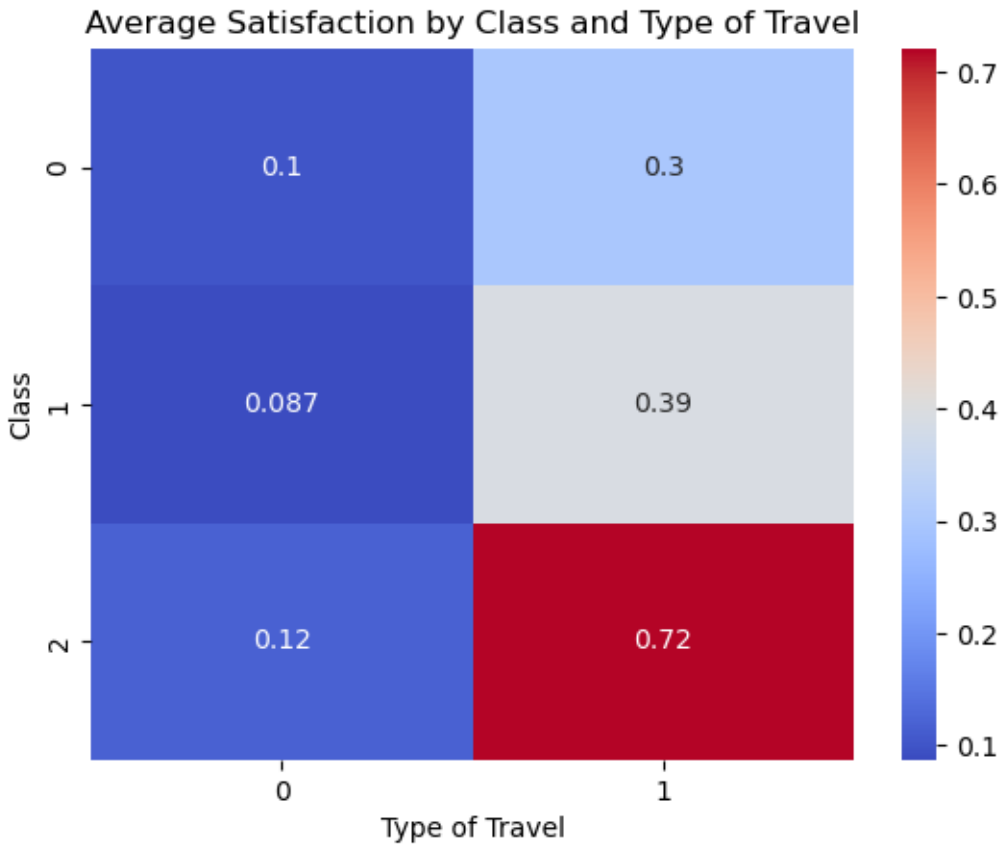
```
satisfaction_by_customer = airlines.groupby("Customer Type")["satisfaction"].
  ↪mean().sort_values(ascending=False)
satisfaction_by_customer.plot(kind="bar", title="Average Satisfaction by␣
  ↪Customer Type", color="lightgreen")

plt.xlabel("Customer Type")
plt.ylabel("Average Satisfaction (0-1)")
plt.show()
```



Satisfaction by Customer Type: Loyal customers score higher on average, consistent with selection/experience effects.

[21]:
```
# Pivot Table (Class vs Type of Travel)
pivot_table = airlines.pivot_table(
    values="satisfaction",
    index="Class",
    columns="Type of Travel",
    aggfunc="mean"
)
pivot_table
```

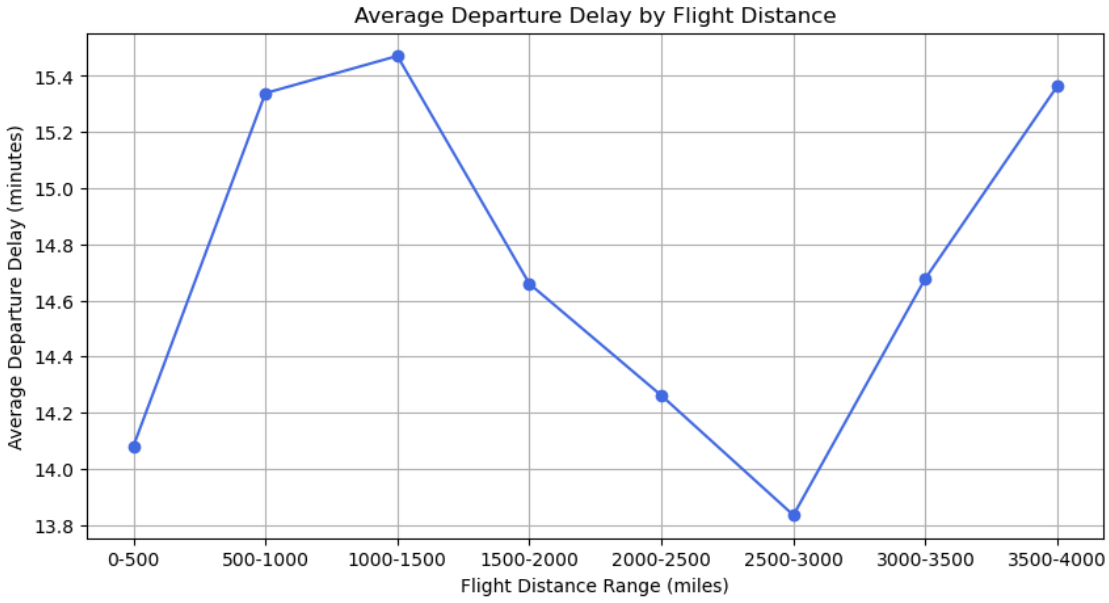
```
sns.heatmap(pivot_table, annot=True, cmap="coolwarm")
plt.title("Average Satisfaction by Class and Type of Travel")
plt.show()
```



Satisfaction is strongest where Business class intersects Business travel, highlighting joint effects of service tier and trip purpose.

[22]:
```
# Aggregate Average Departure Delay by Distance Range
# Create distance bins
bins = [0, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000]
labels = ['0-500', '500-1000', '1000-1500', '1500-2000',
          '2000-2500', '2500-3000', '3000-3500', '3500-4000']

airlines['DistanceGroup'] = pd.cut(
    airlines['Flight Distance_Adjusted'],
    bins=bins, labels=labels, include_lowest=True
)

# Calculate average departure delay per distance group
```

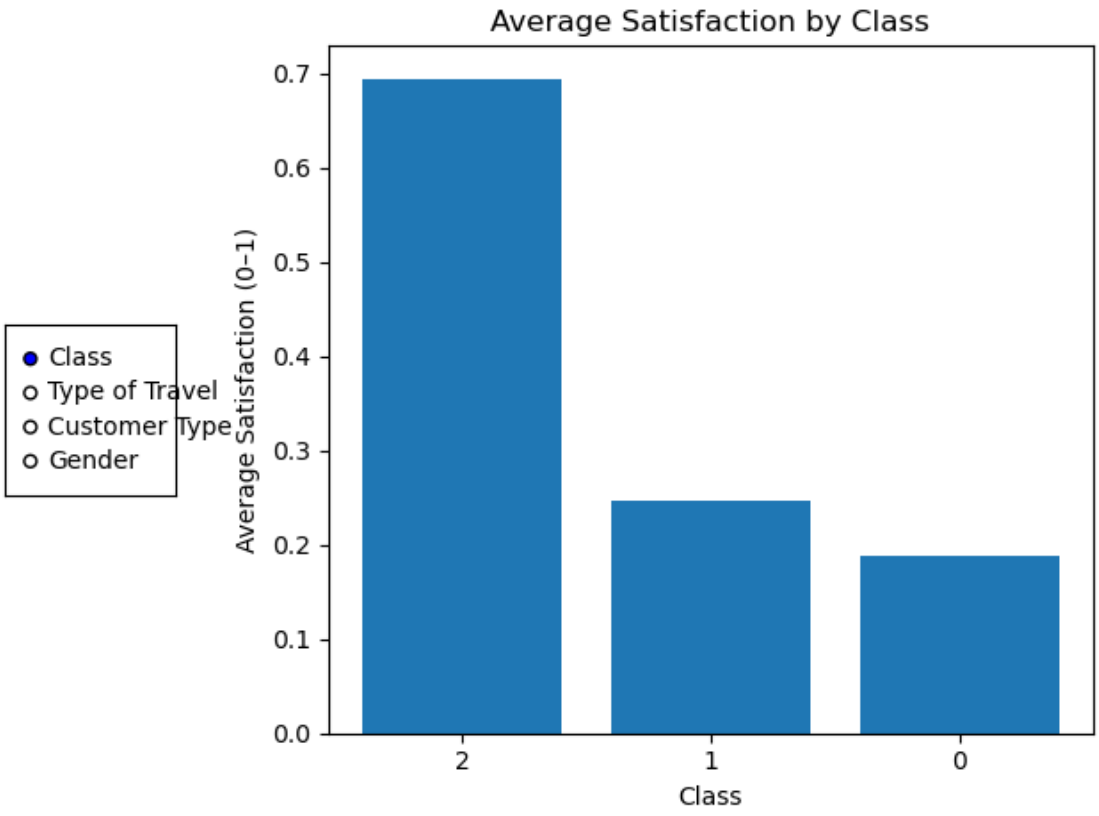
```
avg_delay_by_distance = airlines.groupby("DistanceGroup",␣
 ↪observed=True)["Departure Delay in Minutes"].mean()
avg_delay_by_distance

plt.figure(figsize=(10,5))
avg_delay_by_distance.plot(kind='line', marker='o', color='royalblue')
plt.title("Average Departure Delay by Flight Distance")
plt.xlabel("Flight Distance Range (miles)")
plt.ylabel("Average Departure Delay (minutes)")
plt.grid(True)
plt.show()
```



Delays vary across distance bins (non-monotonic); mid-range bins appear elevated, suggesting operational constraints by route length.

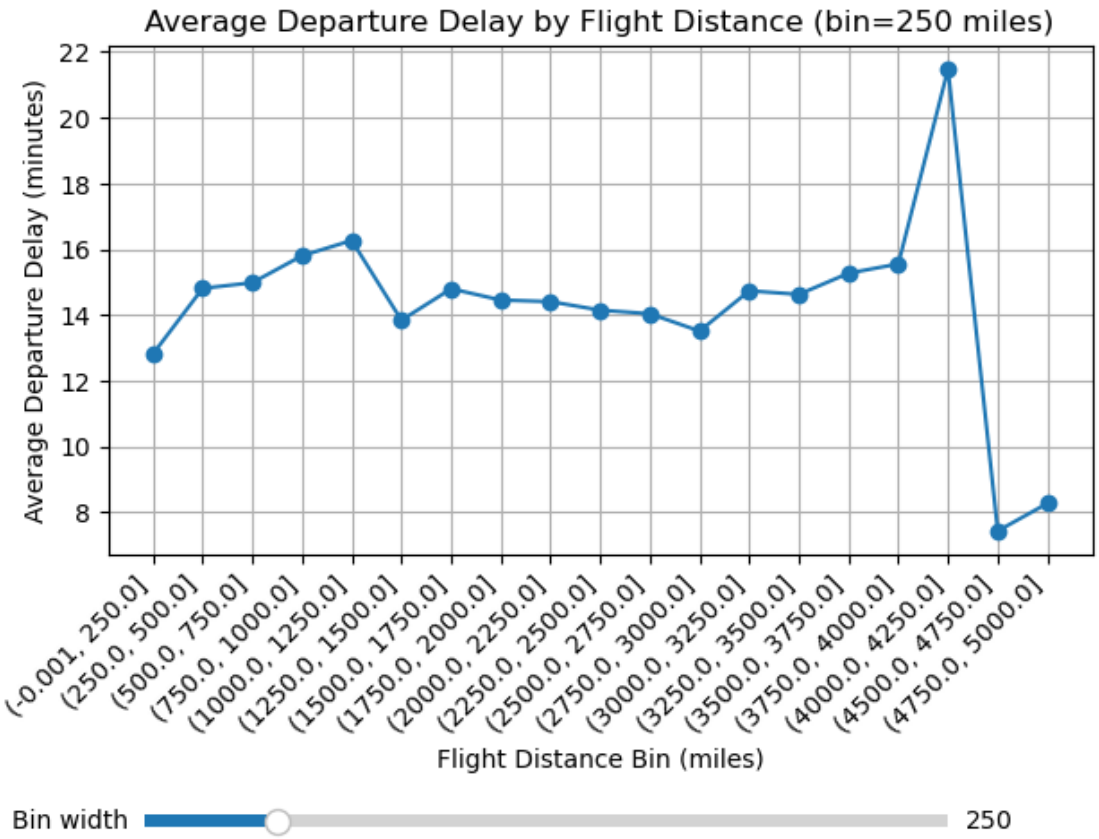
[23]:
```
cat_candidates = [c for c in ['Class', 'Type of Travel', 'Customer Type',␣
 ↪'Gender'] if c in airlines.columns]
if not cat_candidates:
    raise ValueError("Expected categorical columns not found.")

# Initial grouping var
current_group = cat_candidates[0]

# Compute initial means
means = airlines.groupby(current_group)['satisfaction'].mean().
 ↪sort_values(ascending=False)
```

```
fig, ax = plt.subplots()
bars = ax.bar(range(len(means)), means.values)
ax.set_xticks(range(len(means)))
ax.set_xticklabels(means.index.astype(str), rotation=0)
ax.set_title(f"Average Satisfaction by {current_group}")
ax.set_xlabel(current_group)
ax.set_ylabel("Average Satisfaction (0-1)")
plt.tight_layout(rect=[0.2, 0, 1, 1])  # leave space on the left for controls

# RadioButtons on the left
rax = plt.axes([0.02, 0.4, 0.15, 0.2])
radio = RadioButtons(rax, cat_candidates, active=0)

def update_group(label):
    global current_group
    current_group = label
    new_means = airlines.groupby(current_group)['satisfaction'].mean().
 ↪sort_values(ascending=False)
    ax.clear()
    ax.bar(range(len(new_means)), new_means.values)
    ax.set_xticks(range(len(new_means)))
    ax.set_xticklabels(new_means.index.astype(str), rotation=0)
    ax.set_title(f"Average Satisfaction by {current_group}")
    ax.set_xlabel(current_group)
    ax.set_ylabel("Average Satisfaction (0-1)")
    fig.canvas.draw_idle()

radio.on_clicked(update_group)
plt.show()
```



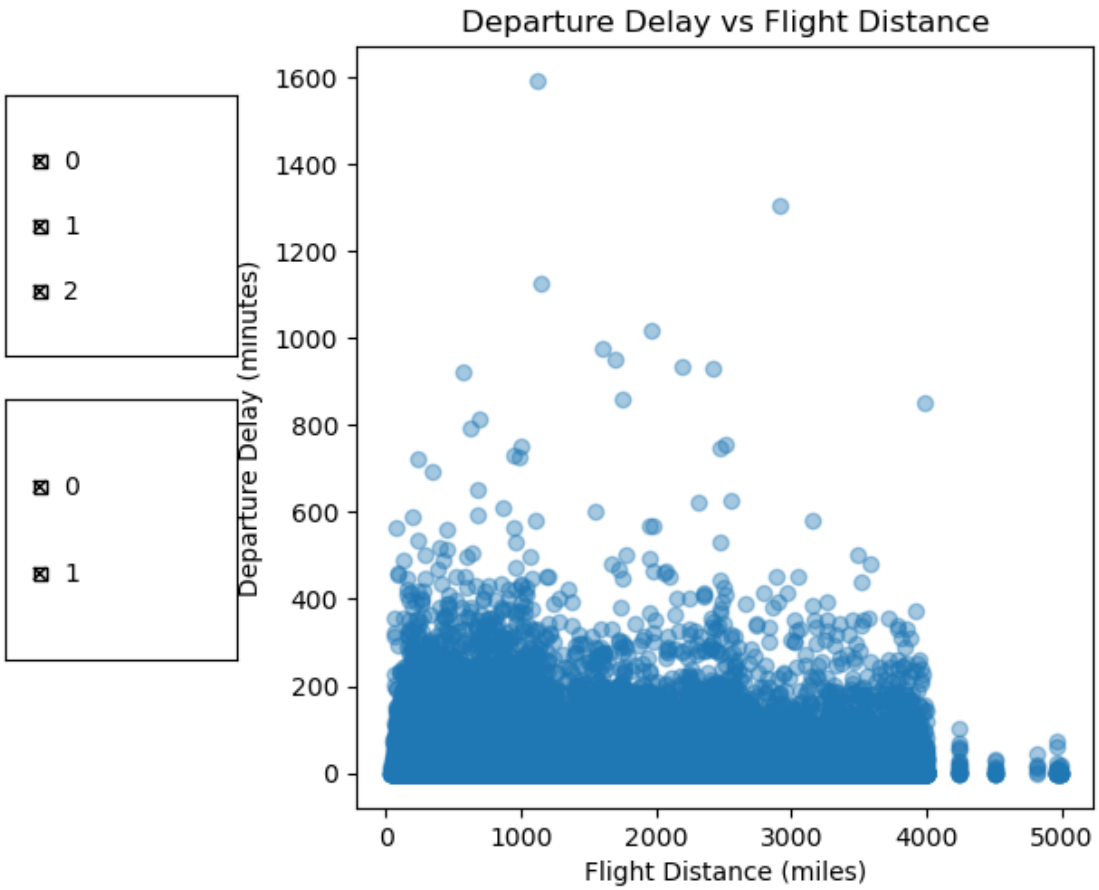
[24]:
```
init_step = 250

dmax = int(max(1, airlines[distance_col].max()))
edges = list(range(0, dmax + init_step, init_step))
grp = pd.cut(airlines[distance_col], bins=edges, include_lowest=True)
avg_delay = airlines.groupby(grp, observed=True)['Departure Delay in Minutes'].
 ↪mean()

fig2, ax2 = plt.subplots()
(line2,) = ax2.plot(range(len(avg_delay)), avg_delay.values, marker='o')
ax2.set_xticks(range(len(avg_delay)))
ax2.set_xticklabels([str(i) for i in avg_delay.index.astype(str)], rotation=45,␣
 ↪ha='right')
ax2.set_title(f"Average Departure Delay by Flight Distance (bin={init_step}␣
 ↪miles)")
ax2.set_xlabel("Flight Distance Bin (miles)")
ax2.set_ylabel("Average Departure Delay (minutes)")
ax2.grid(True)
plt.tight_layout(rect=[0, 0.05, 1, 1])
```

```
# Slider
ax_step = plt.axes([0.15, 0.01, 0.7, 0.03])
step_slider = Slider(ax_step, 'Bin width', 100, 1000, valinit=init_step,␣
 ↪valstep=50)

def on_step_change(val):
    step = int(step_slider.val)
    dmax2 = int(max(1, airlines[distance_col].max()))
    edges2 = list(range(0, dmax2 + step, step))
    grp2 = pd.cut(airlines[distance_col], bins=edges2, include_lowest=True)
    avg2 = airlines.groupby(grp2, observed=True)['Departure Delay in Minutes'].
 ↪mean()
    ax2.clear()
    ax2.plot(range(len(avg2)), avg2.values, marker='o')
    ax2.set_xticks(range(len(avg2)))
    ax2.set_xticklabels([str(i) for i in avg2.index.astype(str)], rotation=45,␣
 ↪ha='right')
    ax2.set_title(f"Average Departure Delay by Flight Distance (bin={step}␣
 ↪miles)")
    ax2.set_xlabel("Flight Distance Bin (miles)")
    ax2.set_ylabel("Average Departure Delay (minutes)")
    ax2.grid(True)
    fig2.canvas.draw_idle()

step_slider.on_changed(on_step_change)
plt.show()
```



[25]:
```
class_vals = sorted(airlines['Class'].dropna().unique().tolist()) if 'Class' in␣
 ↪airlines.columns else []
travel_vals = sorted(airlines['Type of Travel'].dropna().unique().tolist()) if␣
 ↪'Type of Travel' in airlines.columns else []

fig3, ax3 = plt.subplots()
plt.tight_layout(rect=[0.25, 0, 1, 1])  # space on left for controls

# Initial mask: all selected
selected_class = {v: True for v in class_vals}
selected_travel = {v: True for v in travel_vals}

def current_mask():
    df = airlines.copy()
    if class_vals:
        df = df[df['Class'].isin([k for k,v in selected_class.items() if v])]
    if travel_vals:
        df = df[df['Type of Travel'].isin([k for k,v in selected_travel.items()␣
 ↪if v])]
    return df
```

```
def redraw():
    df = current_mask()
    ax3.clear()
    ax3.scatter(df[distance_col], df['Departure Delay in Minutes'], alpha=0.4)
    ax3.set_title("Departure Delay vs Flight Distance")
    ax3.set_xlabel("Flight Distance (miles)")
    ax3.set_ylabel("Departure Delay (minutes)")
    fig3.canvas.draw_idle()

redraw()

# Controls: two groups of CheckButtons
y0 = 0.6
if class_vals:
    rax_class = plt.axes([0.02, y0, 0.2, 0.3])
    labels_class = [str(v) for v in class_vals]
    checks_class = CheckButtons(rax_class, labels_class, [True]*len(class_vals))
    def on_class(label):
        key = int(label) if label.isdigit() else label
        selected_class[key] = not selected_class[key]
        redraw()
    checks_class.on_clicked(on_class)
    y0 -= 0.35

if travel_vals:
    rax_travel = plt.axes([0.02, y0, 0.2, 0.3])
    labels_travel = [str(v) for v in travel_vals]
    checks_travel = CheckButtons(rax_travel, labels_travel,␣
 ↪[True]*len(travel_vals))
    def on_travel(label):
        key = int(label) if label.isdigit() else label
        selected_travel[key] = not selected_travel[key]
        redraw()
    checks_travel.on_clicked(on_travel)

plt.show()
```



3.0.7 Results

Across ~130k records, service/flight factors relate meaningfully to outcomes. Business class yields the highest satisfaction; loyal customers rate experiences higher than disloyal customers. Satisfaction peaks for Business class on Business trips, indicating a joint effect of cabin and purpose. Average departure delays vary by distance group, with mid-range distances showing elevated means relative to the shortest and longest groups. These patterns support H and H and suggest airlines can improve satisfaction by aligning service tier to trip purpose and address delay hot-spots in specific distance bands. Limitations: the dataset lacks dates/airports (no external drivers like weather or traffic), so causality can't be inferred; results describe associations within the available variables.

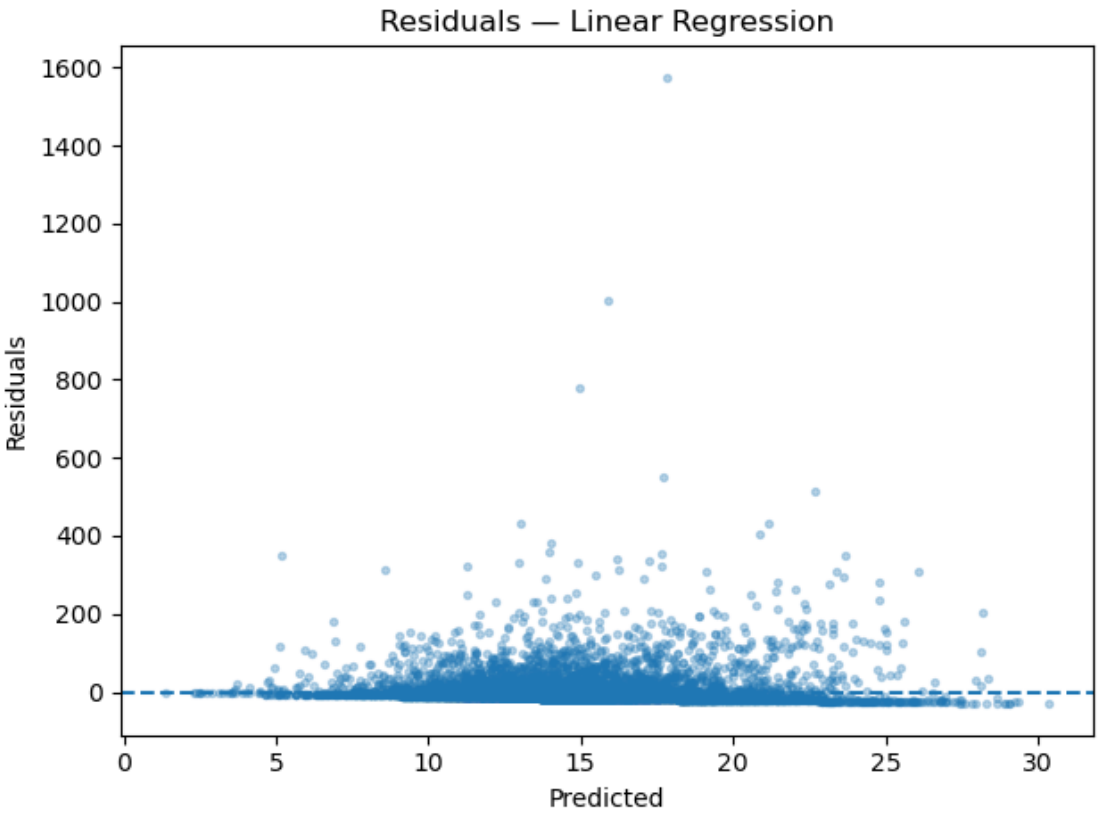
4 Modeling

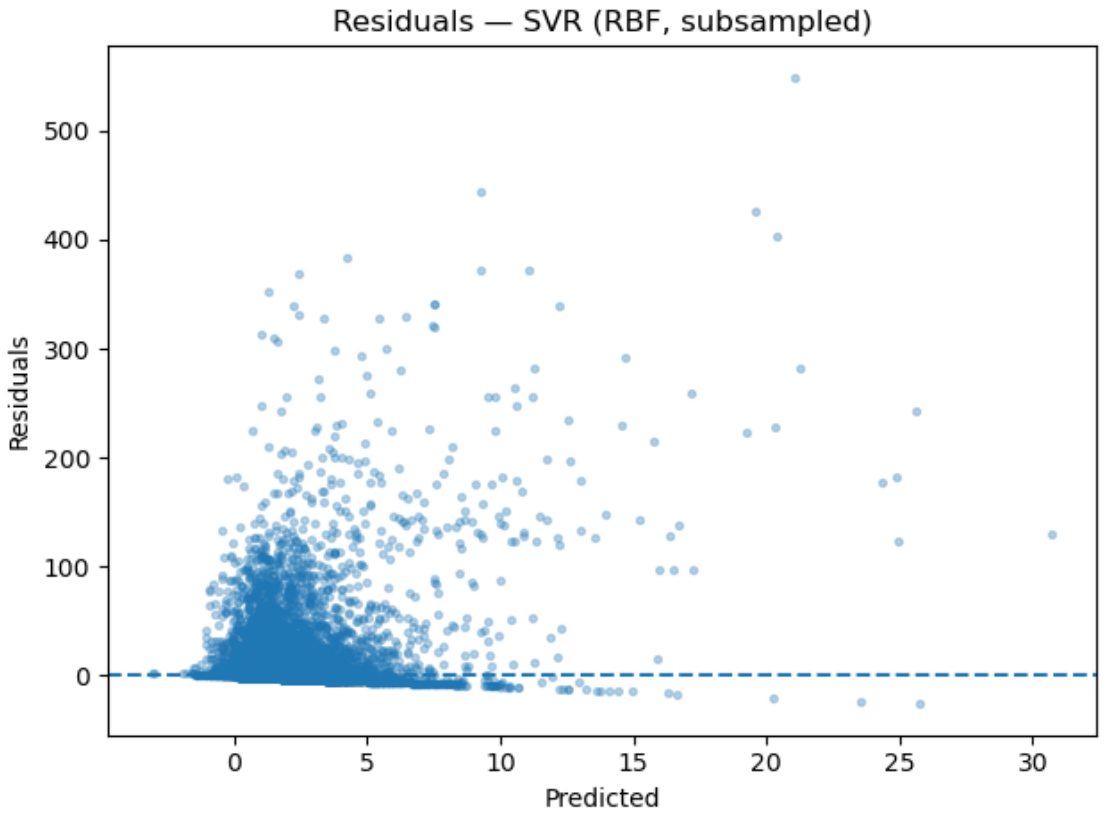
[26]:
```
# define the target
y = airlines["Departure Delay in Minutes"].astype(float)
distance_col = "Flight Distance_Adjusted" if "Flight Distance_Adjusted" in␣
 ↪airlines.columns else "Flight Distance"
```

```
feature_cols = [
    "Age", distance_col,
    "Inflight wifi service", "Departure/Arrival time convenient", "Ease of␣
 ↪Online booking",
    "Gate location", "Food and drink", "Online boarding", "Seat comfort",
    "Inflight entertainment", "On-board service", "Leg room service", "Baggage␣
 ↪handling",
    "Checkin service", "Inflight service", "Cleanliness",
    "Gender", "Customer Type", "Type of Travel", "Class"
]
X = airlines[feature_cols].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

def report(y_true, y_pred, label):
    rmse = root_mean_squared_error(y_true, y_pred)
    mae  = mean_absolute_error(y_true, y_pred)
    r2   = r2_score(y_true, y_pred)
    print(f"{label:>16} | RMSE={rmse:.3f}  MAE={mae:.3f}  R²={r2:.3f}")

# baseline and linear regression
t0 = time.time()
baseline = Pipeline([("scaler", StandardScaler()), ("model",␣
 ↪DummyRegressor(strategy="mean"))])
baseline.fit(X_train, y_train)
pred_base = baseline.predict(X_test)
report(y_test, pred_base, "Baseline (Mean)")
print(f"Baseline done in {time.time()-t0:.2f}s")

t1 = time.time()
linreg = Pipeline([("scaler", StandardScaler()), ("model", LinearRegression())])
linreg.fit(X_train, y_train)
pred_lr = linreg.predict(X_test)
report(y_test, pred_lr, "Linear Reg.")
cv_lr = -cross_val_score(linreg, X_train, y_train, cv=5,␣
 ↪scoring="neg_root_mean_squared_error")
print(f"Linear Reg. CV RMSE: {cv_lr.mean():.3f} ± {cv_lr.std():.3f}")
print(f"Linear Reg. done in {time.time()-t1:.2f}s")

# subsample SVR
SVR_TRAIN_CAP = 12_000
rng = np.random.default_rng(42)
if len(X_train) > SVR_TRAIN_CAP:
    idx = rng.choice(len(X_train), size=SVR_TRAIN_CAP, replace=False)
```

```
    Xs, ys = X_train.iloc[idx], y_train.iloc[idx]
else:
    Xs, ys = X_train, y_train

t2 = time.time()
svr = Pipeline([("scaler", StandardScaler()),
                ("model", SVR(kernel="rbf", C=10.0, epsilon=1.0, gamma="scale",␣
 ↪cache_size=500, tol=1e-3))])
svr.fit(Xs, ys)
pred_svr = svr.predict(X_test)
report(y_test, pred_svr, f"SVR (RBF, n={len(Xs)})")
print(f"SVR fit on {len(Xs)} rows in {time.time()-t2:.2f}s")

# residual plots
def plot_residuals_fast(y_true, y_pred, title, sample=10000):
    n = len(y_pred)
    if sample and n > sample:
        idx = rng.choice(n, size=sample, replace=False)
    else:
        idx = np.arange(n)
    y_t = y_true.to_numpy() if hasattr(y_true, "to_numpy") else np.
 ↪asarray(y_true)
    p   = np.asarray(y_pred)
    resid = y_t[idx] - p[idx]
    fig, ax = plt.subplots()
    ax.scatter(p[idx], resid, alpha=0.3, s=8)
    ax.axhline(0, linestyle="--")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Residuals")
    ax.set_title(title)
    fig.tight_layout()
    plt.show()
    plt.close(fig)

plot_residuals_fast(y_test, pred_lr,  "Residuals - Linear Regression",␣
 ↪sample=10000)
plot_residuals_fast(y_test, pred_svr, "Residuals - SVR (RBF, subsampled)",␣
 ↪sample=10000)
```

```
 Baseline (Mean) | RMSE=37.921  MAE=20.081  R²=-0.000
Baseline done in 0.05s
     Linear Reg. | RMSE=37.835  MAE=20.096  R²=0.005
Linear Reg. CV RMSE: 37.934 ± 1.490
Linear Reg. done in 0.51s
SVR (RBF, n=12000) | RMSE=39.442  MAE=14.850  R²=-0.082
SVR fit on 12000 rows in 23.64s
```

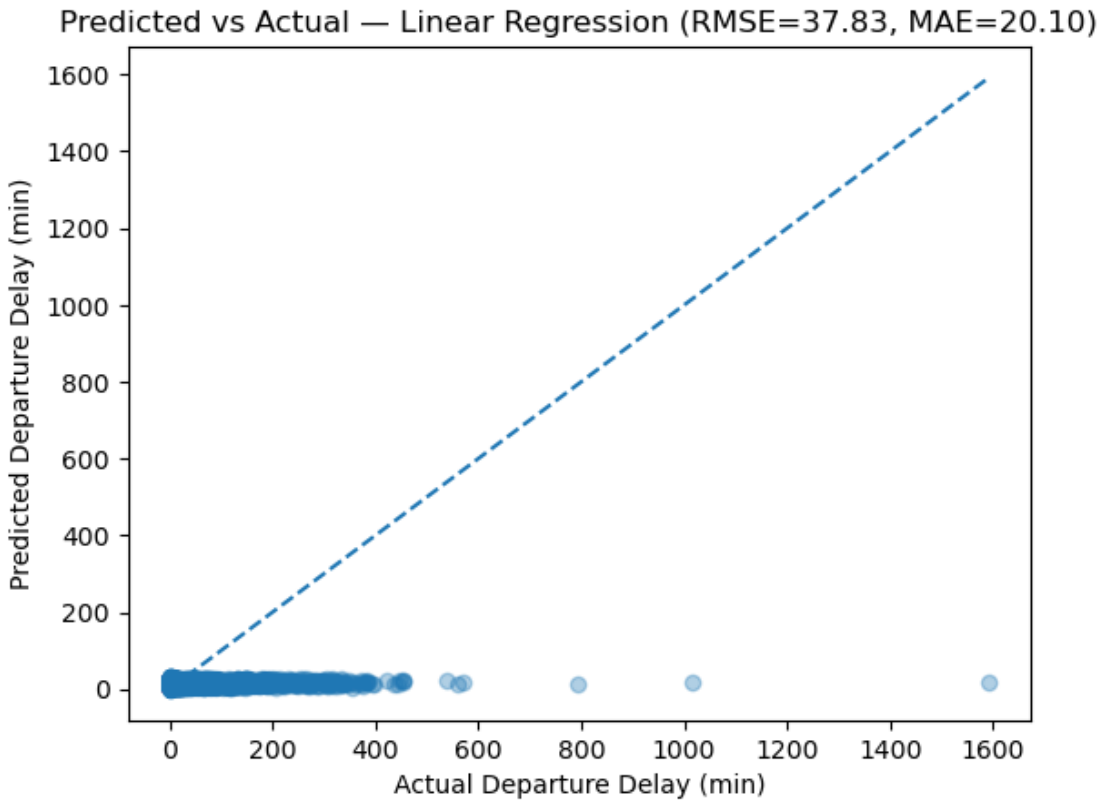

Residuals — Linear Regression
Residuals
Predicted
1600
1400
1200
1000
800
600
400
200
0
0
5
10
15
20
25
30

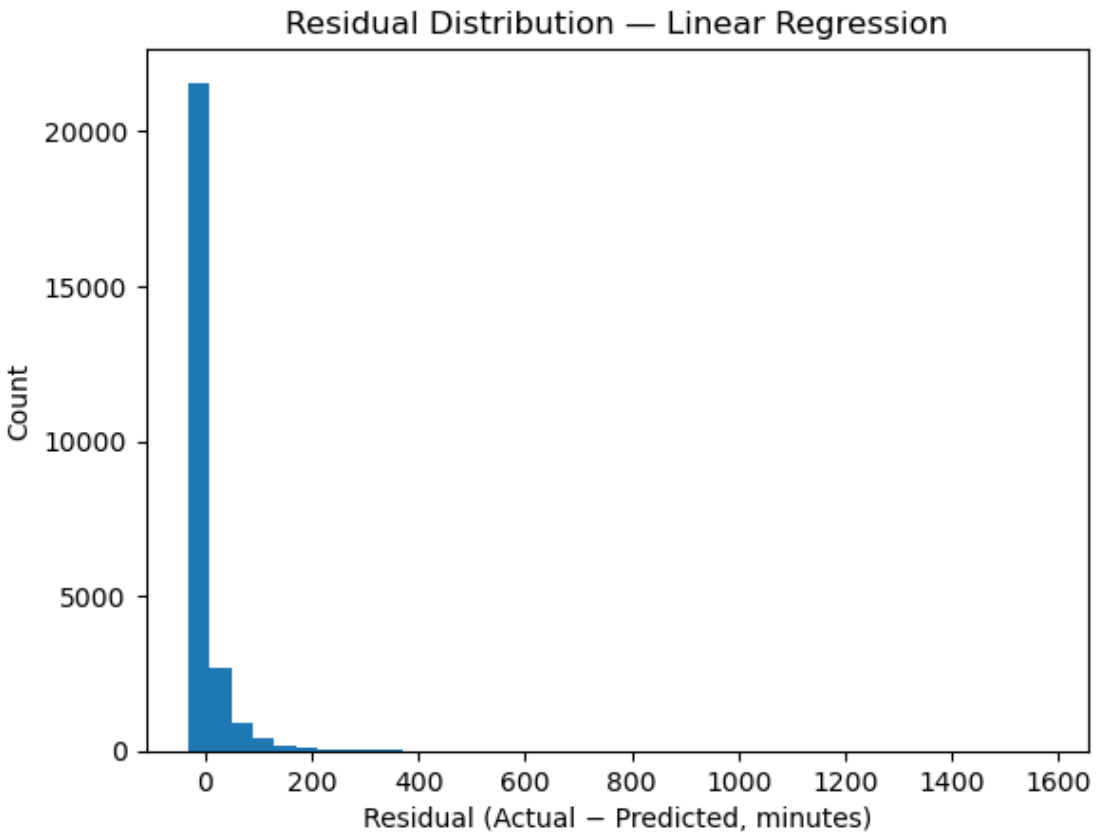


Training the Linear Regression model and SVR to predict the Departure Delay using demographic and service rating features. Both models performed about the same as a mean-predicteion baseline. Predicted vs actual scatter was diffuse and residuals were centered without strong patterns, indicating the models aren't systematically biased but simply lack signal.

[27]:
```
pred_lr  = linreg.predict(X_test)
pred_svr = svr.predict(X_test)

rmse_lr  = root_mean_squared_error(y_test, pred_lr)
rmse_svr = root_mean_squared_error(y_test, pred_svr)

best_name, best_pred, best_rmse = (
    ("Linear Regression", pred_lr, rmse_lr) if rmse_lr <= rmse_svr else ("SVR␣
 ↪(RBF)", pred_svr, rmse_svr)
)
mae = mean_absolute_error(y_test, best_pred)

# predicted vs Actual
plt.figure()
plt.scatter(y_test, best_pred, alpha=0.35)
lo = float(np.min([y_test.min(), best_pred.min()]))
```

```
hi = float(np.max([y_test.max(), best_pred.max()]))
plt.plot([lo, hi], [lo, hi], linestyle="--")
plt.xlabel("Actual Departure Delay (min)")
plt.ylabel("Predicted Departure Delay (min)")
plt.title(f"Predicted vs Actual - {best_name} (RMSE={best_rmse:.2f}, MAE={mae:.
  ↪2f})")
plt.show()

# residual distribution
resid = y_test - best_pred
plt.figure()
plt.hist(resid, bins=40)
plt.xlabel("Residual (Actual - Predicted, minutes)")
plt.ylabel("Count")
plt.title(f"Residual Distribution - {best_name}")
plt.show()

print(f"Best model: {best_name} | RMSE={best_rmse:.3f} | MAE={mae:.3f}")
```






```
Best model: Linear Regression | RMSE=37.835 | MAE=20.096
```

5 Arrival Delay Model LR vs SVR

[28]:
```
# Target & features
yA = airlines["Arrival Delay in Minutes"].astype(float)
distance_col = "Flight Distance_Adjusted" if "Flight Distance_Adjusted" in␣
 ↪airlines.columns else "Flight Distance"
num_colsA = ["Departure Delay in Minutes", distance_col, "Age"]
cat_colsA = [c for c in ["Type of Travel", "Class", "Customer Type", "Gender"]␣
 ↪if c in airlines.columns]
XA = airlines[num_colsA + cat_colsA].copy()
print("Targets complete")

# Train/test split
XA_train, XA_test, yA_train, yA_test = train_test_split(XA, yA, test_size=0.2,␣
 ↪random_state=42)

print("Train and Test complete")
```

```
# Preprocess
preA = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_colsA),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"),␣
 ↪cat_colsA),
    ],
    remainder="drop",
)

print("Preprocessing complete")

def reportA(y_true, y_pred, label):
    rmse = root_mean_squared_error(y_true, y_pred)
    mae  = mean_absolute_error(y_true, y_pred)
    r2   = r2_score(y_true, y_pred)
    print(f"{label:>18} | RMSE={rmse:.3f} MAE={mae:.3f} R²={r2:.3f}")
    return rmse, mae, r2

results = []

print("Results Started")

# baseline and Ridge
baseA  = Pipeline([("pre", preA), ("m", DummyRegressor(strategy="mean"))]).
 ↪fit(XA_train, yA_train)
pred_b = baseA.predict(XA_test); results.append(("Baseline",) +␣
 ↪reportA(yA_test, pred_b, "Baseline (Mean)") + (float("nan"),))

linA   = Pipeline([("pre", preA), ("m", LinearRegression())]).fit(XA_train,␣
 ↪yA_train)
pred_l = linA.predict(XA_test);  results.append(("Linear",) + reportA(yA_test,␣
 ↪pred_l, "Linear Regression") + (float("nan"),))

ridgeA = Pipeline([("pre", preA), ("m", Ridge(alpha=3.0))]).fit(XA_train,␣
 ↪yA_train)
pred_r = ridgeA.predict(XA_test); results.append(("Ridge  =3",) +␣
 ↪reportA(yA_test, pred_r, "Ridge  =3.0") + (float("nan"),))

print("Baseline complete")

# SVR RBF subsample and small grid
SVR_CAP = 12000
rng = np.random.default_rng(42)
if len(XA_train) > SVR_CAP:
```

```
    take = rng.choice(len(XA_train), size=SVR_CAP, replace=False)
    Xs, ys = XA_train.iloc[take], yA_train.iloc[take]
else:
    Xs, ys = XA_train, yA_train

svr_pipe = Pipeline([
    ("pre", preA),
    ("m", SVR(kernel="rbf", cache_size=500, tol=1e-3))
])

print("SVR complete")

# a tiny, sane grid
param_grid = {
    "m__C":       [3, 10],
    "m__epsilon": [0.1, 0.5],
    "m__gamma":   ["scale", 0.01],
}

grid = GridSearchCV(
    svr_pipe,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=3,
    n_jobs=1,      # keep 1 to avoid certain Windows/kernel stalls
    verbose=2
)

print("Grid Data complete")

grid.fit(Xs, ys)
pred_svr = grid.predict(XA_test)
rmse, mae, r2 = reportA(yA_test, pred_svr, "SVR best (subsampled)")
results.append((f"SVR best (n={len(Xs)})", rmse, mae, r2, -float(grid.
 ↪best_score_)))

print("Results complete")

# generate a comparison table
comp = pd.DataFrame(results, columns=["Model","RMSE","MAE","R2","CV_RMSE␣
 ↪(3-fold)"]).sort_values("RMSE")
print("\nModel comparison:")
print(comp.to_string(index=False))
```

```
Targets complete
Train and Test complete
Preprocessing complete
```

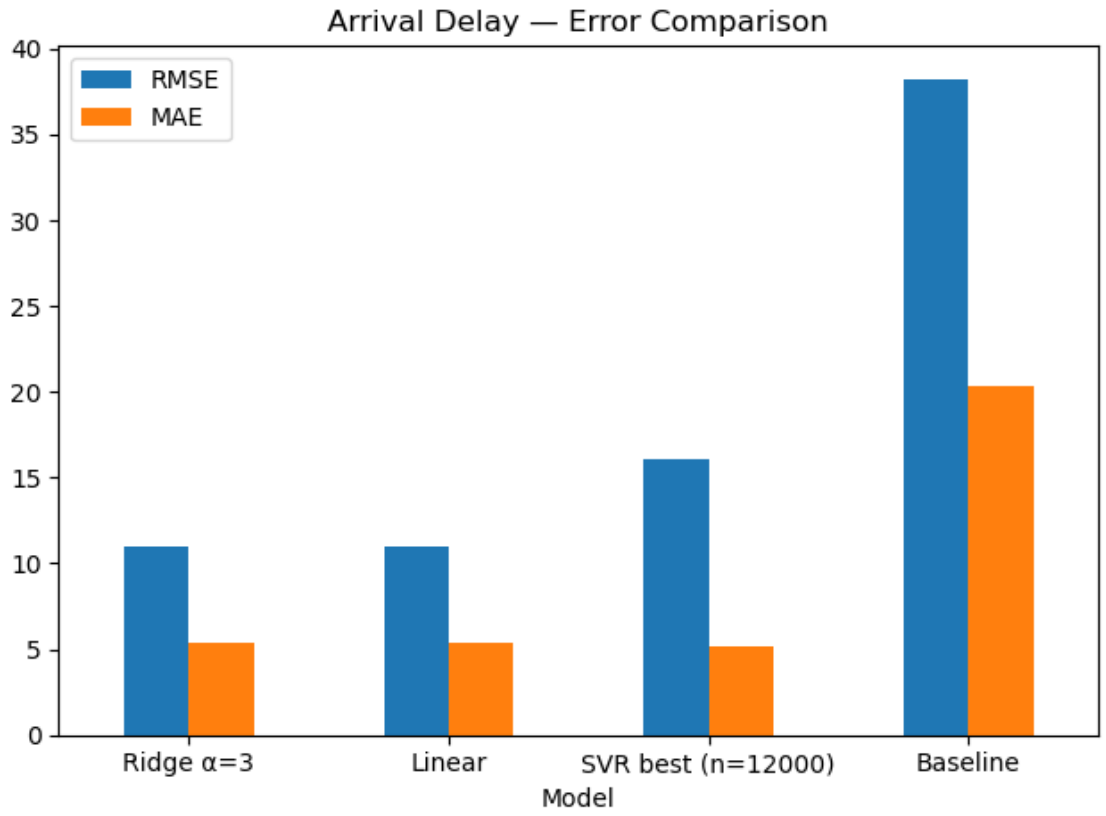
```
Results Started
   Baseline (Mean) | RMSE=38.240 MAE=20.362 R²=-0.000
 Linear Regression | RMSE=10.973 MAE=5.367 R²=0.918
       Ridge  =3.0 | RMSE=10.973 MAE=5.367 R²=0.918
Baseline complete
SVR complete
Grid Data complete
Fitting 3 folds for each of 8 candidates, totalling 24 fits
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=scale; total time=   3.2s
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=scale; total time=   3.3s
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=scale; total time=   3.2s
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=0.01; total time=   3.2s
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=0.01; total time=   3.0s
[CV] END …m__C=3, m__epsilon=0.1, m__gamma=0.01; total time=   2.9s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=scale; total time=   3.1s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=scale; total time=   3.1s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=scale; total time=   2.7s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=0.01; total time=   2.5s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=0.01; total time=   2.7s
[CV] END …m__C=3, m__epsilon=0.5, m__gamma=0.01; total time=   2.7s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=scale; total time=   3.9s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=scale; total time=   4.1s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=scale; total time=   3.9s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=0.01; total time=   2.9s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=0.01; total time=   3.0s
[CV] END …m__C=10, m__epsilon=0.1, m__gamma=0.01; total time=   2.9s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=scale; total time=   2.8s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=scale; total time=   3.0s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=scale; total time=   3.1s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=0.01; total time=   2.7s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=0.01; total time=   3.0s
[CV] END …m__C=10, m__epsilon=0.5, m__gamma=0.01; total time=   2.7s
SVR best (subsampled) | RMSE=16.053 MAE=5.122 R²=0.824
Results complete

Model comparison:
              Model      RMSE       MAE        R2  CV_RMSE (3-fold)
         Ridge  =3 10.973380  5.367395  0.917655               NaN
            Linear 10.973390  5.367251  0.917655               NaN
SVR best (n=12000) 16.052659  5.121787  0.823781         14.747794
          Baseline 38.240318 20.361591 -0.000001               NaN
```

5.0.1 K-fold cross validation for estimating how well the model will generalize

[29]:
```
cv = KFold(n_splits=5, shuffle=True, random_state=42)

lin_cv = -cross_val_score(
```

```
    Pipeline([("pre", preA), ("m", LinearRegression())]),
    XA_train, yA_train, scoring="neg_root_mean_squared_error", cv=cv
).mean()

ridge_cv = -cross_val_score(
    Pipeline([("pre", preA), ("m", Ridge(alpha=3.0))]),
    XA_train, yA_train, scoring="neg_root_mean_squared_error", cv=cv
).mean()

comp.loc[comp["Model"]=="Linear", "CV_RMSE (3-fold)"] = round(lin_cv, 6)
comp.loc[comp["Model"]=="Ridge  =3", "CV_RMSE (3-fold)"] = round(ridge_cv, 6)
print(comp.to_string(index=False))

for a in [0.1, 1, 3, 10, 30]:
    mdl = Pipeline([("pre", preA), ("m", Ridge(alpha=a))]).fit(XA_train,␣
 ↪yA_train)
    pred = mdl.predict(XA_test)
    print(f"Ridge  ={a:<4} | RMSE={root_mean_squared_error(yA_test, pred):.3f}")

ax = comp.set_index("Model")[["RMSE","MAE"]].plot.bar(rot=0)
plt.title("Arrival Delay - Error Comparison")
plt.tight_layout(); plt.show()
```

```
             Model      RMSE       MAE        R2  CV_RMSE (3-fold)
         Ridge  =3 10.973380  5.367395  0.917655         10.798296
            Linear 10.973390  5.367251  0.917655         10.798296
SVR best (n=12000) 16.052659  5.121787  0.823781         14.747794
          Baseline 38.240318 20.361591 -0.000001               NaN
Ridge  =0.1  | RMSE=10.973
Ridge  =1    | RMSE=10.973
Ridge  =3    | RMSE=10.973
Ridge  =10   | RMSE=10.973
Ridge  =30   | RMSE=10.973
```



5.1 Findings - Arrival Delay

Ridge (= 3) and Linear Regression performed best and essentially identically (RMSE 11.0, MAE 5.37, R squared 0.918). Five-Fold cross validation for Ridge produced CV RMSE 1.08, closely matching test error and indicating good generalization. The tuned RBF-SVR underperformed despite a similar MAE, suggesting the target is well modeled by a linear relationship. The baseline was far worse. Conclusion: Select Ridge (= 3) as the final model for stability and regularization.

5.2 Linear Ridge Model

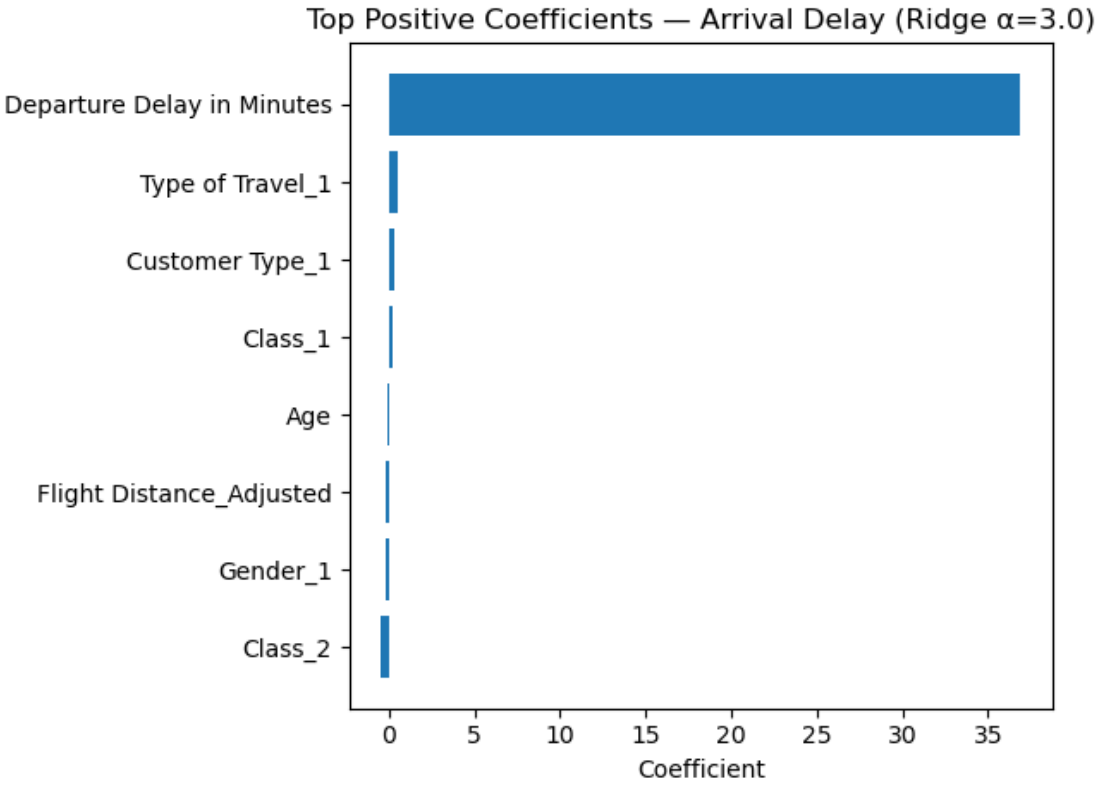
5.2.1 Find which features most strongly increase or decrease the Arrival Delay in minutes.

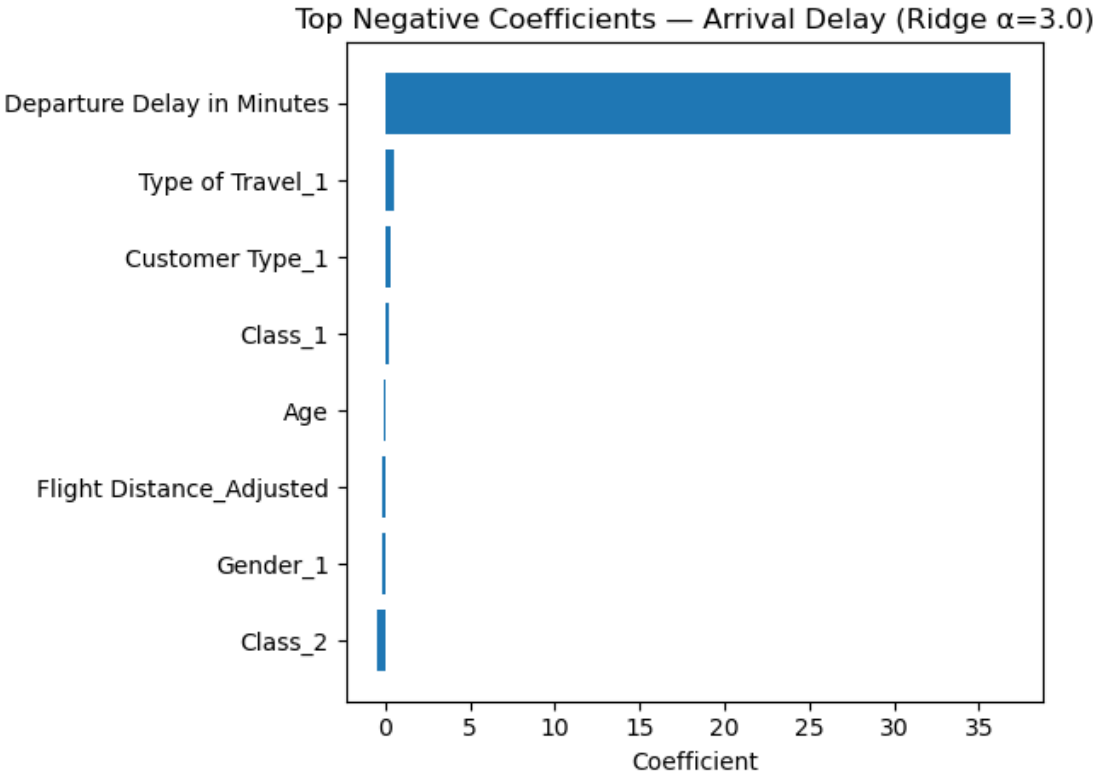
[30]:
```
def _coef_table_from_fitted_linear_pipeline(fitted_pipe):
    pre = fitted_pipe.named_steps["pre"]
    feat_names = pre.get_feature_names_out()
    coefs = fitted_pipe.named_steps["m"].coef_.astype(float)
    df = (pd.DataFrame({"feature": feat_names, "coef": coefs})
            .assign(abs_coef=lambda d: d["coef"].abs()))
    df["feature_clean"] = df["feature"].str.replace(r"^(num|cat)__", "",␣
↪regex=True)
```

```
    return df.sort_values("abs_coef", ascending=False)

fitted_for_coefs = None

# prefer your existing fitted Ridge model if present
try:
    if "ridgeA" in globals() and hasattr(ridgeA, "named_steps"):
        fitted_for_coefs = ridgeA
except NameError:
    pass

# fall back to temp if not available
if fitted_for_coefs is None:
    from sklearn.pipeline import Pipeline
    from sklearn.linear_model import Ridge
    _ridge_temp = Pipeline([("pre", preA), ("m", Ridge(alpha=3.0))]).
 ↪fit(XA_train, yA_train)
    fitted_for_coefs = _ridge_temp

# coefficient table
arrival_coef = _coef_table_from_fitted_linear_pipeline(fitted_for_coefs)

# Show top drivers
top_pos = arrival_coef.sort_values("coef", ascending=False).head(10).copy()
top_neg = arrival_coef.sort_values("coef", ascending=True).head(10).copy()

print("\nTop + drivers of Arrival Delay (Ridge  =3.0):")
print(top_pos[["feature_clean","coef"]].to_string(index=False, justify="left",␣
 ↪float_format=lambda x: f"{x: .4f}"))

print("\nTop - drivers of Arrival Delay (Ridge  =3.0):")
print(top_neg[["feature_clean","coef"]].to_string(index=False, justify="left",␣
 ↪float_format=lambda x: f"{x: .4f}"))

# Simple bar charts
plt.figure()
plt.barh(top_pos["feature_clean"][::-1], top_pos["coef"][::-1])
plt.title("Top Positive Coefficients - Arrival Delay (Ridge  =3.0)")
plt.xlabel("Coefficient"); plt.tight_layout(); plt.show()

plt.figure()
plt.barh(top_neg["feature_clean"], top_neg["coef"])
plt.title("Top Negative Coefficients - Arrival Delay (Ridge  =3.0)")
plt.xlabel("Coefficient"); plt.tight_layout(); plt.show()

# Clean up the temporary model if we created it
try:
```

```
    del _ridge_temp
except NameError:
    pass
```

```
Top + drivers of Arrival Delay (Ridge  =3.0):
feature_clean               coef
Departure Delay in Minutes  36.9067
          Type of Travel_1   0.4904
           Customer Type_1   0.3202
                  Class_1   0.1815
                      Age  -0.1177
  Flight Distance_Adjusted  -0.1385
                 Gender_1  -0.1409
                  Class_2  -0.4570

Top - drivers of Arrival Delay (Ridge  =3.0):
feature_clean               coef
                  Class_2  -0.4570
                 Gender_1  -0.1409
  Flight Distance_Adjusted  -0.1385
                      Age  -0.1177
                  Class_1   0.1815
           Customer Type_1   0.3202
          Type of Travel_1   0.4904
Departure Delay in Minutes  36.9067
```


Top Positive Coefficients — Arrival Delay (Ridge α=3.0)
Departure Delay in Minutes
Type of Travel_1
Customer Type_1
Class_1
Age
Flight Distance_Adjusted
Gender_1
Class_2
0
5
10
15
20
25
30
35
Coefficient

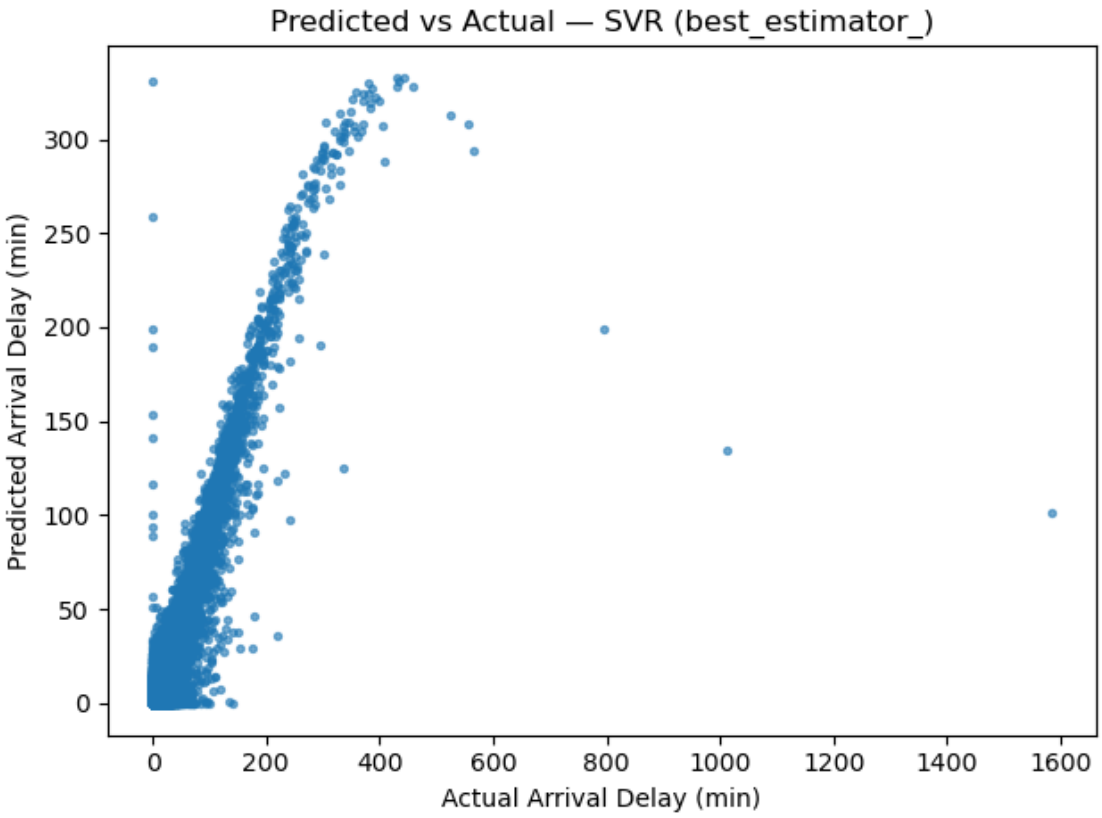

Top Negative Coefficients — Arrival Delay (Ridge α=3.0)
Departure Delay in Minutes
Type of Travel_1
Customer Type_1
Class_1
Age
Flight Distance_Adjusted
Gender_1
Class_2
0
5
10
15
20
25
30
35
Coefficient

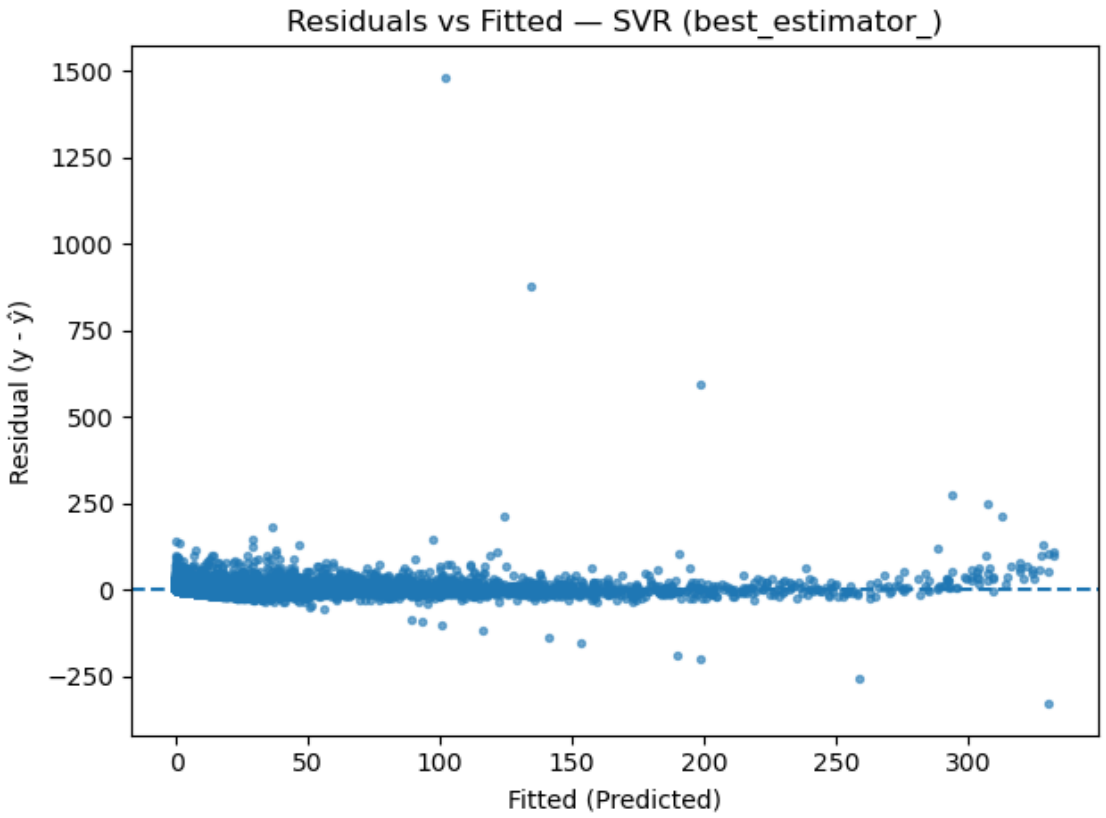
5.3 Residual Diag + Predicted vs Actual Values

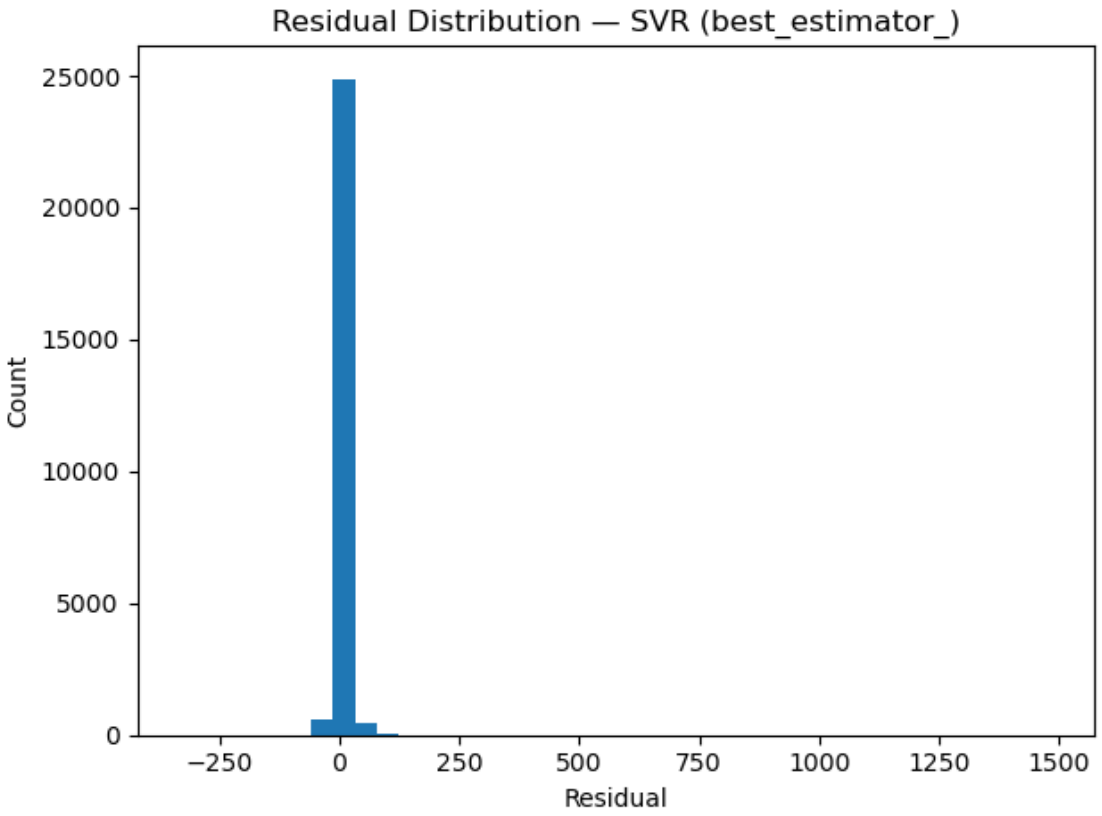
[31]:
```
def _pick_regressor():
    try:
        if 'best_reg' in globals():
            if best_reg == 'svr' and 'grid' in globals():
                return grid.best_estimator_, "SVR (best_estimator_)"
            if best_reg == 'ridge' and 'ridgeA' in globals():
                return ridgeA, "Ridge"
            if best_reg == 'linear' and 'linA' in globals():
                return linA, "Linear Regression"
    except Exception:
        pass

    # Otherwise, pick the first available in a sensible order
    if 'grid' in globals():
        try:
            return grid.best_estimator_, "SVR (best_estimator_)"
        except Exception:
            pass
    if 'ridgeA' in globals():
        return ridgeA, "Ridge"
    if 'linA' in globals():
        return linA, "Linear Regression"

    raise RuntimeError("No fitted regressor found. Expected one of: grid.
 ↪best_estimator_, ridgeA, or linA.")

# Pick model & predict
reg_model, reg_label = _pick_regressor()
y_pred = reg_model.predict(XA_test)
resid  = yA_test - y_pred

# Metrics
rmse = root_mean_squared_error(yA_test, y_pred)
mae  = mean_absolute_error(yA_test, y_pred)
r2   = r2_score(yA_test, y_pred)
print(f"[{reg_label}] RMSE={rmse:.3f}  MAE={mae:.3f}  R²={r2:.3f}")

# Save for slides/table
pred_df = pd.DataFrame({
    "y_true": np.asarray(yA_test),
    "y_pred": np.asarray(y_pred),
    "residual": np.asarray(resid),
```

```
})


# --- Plot 1: Predicted vs Actual (45° line) ---
plt.figure()
plt.scatter(yA_test, y_pred, s=8, alpha=0.6)
min_ax = float(np.percentile(np.r_[yA_test, y_pred], 1))
max_ax = float(np.percentile(np.r_[yA_test, y_pred], 99))
plt.plot([min_ax, max_ax], [min_ax, max_ax])
plt.xlabel("Actual Arrival Delay (min)")
plt.ylabel("Predicted Arrival Delay (min)")
plt.title(f"Predicted vs Actual - {reg_label}")
plt.tight_layout()
plt.show()

# --- Plot 2: Residuals vs Fitted ---
plt.figure()
plt.scatter(y_pred, resid, s=8, alpha=0.6)
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted (Predicted)")
plt.ylabel("Residual (y - ŷ)")
plt.title(f"Residuals vs Fitted - {reg_label}")
plt.tight_layout()
plt.show()

# --- Plot 3: Residual distribution (histogram) ---
plt.figure()
plt.hist(resid, bins=40)
plt.xlabel("Residual")
plt.ylabel("Count")
plt.title(f"Residual Distribution - {reg_label}")
plt.tight_layout()
plt.show()
```

```
[SVR (best_estimator_)] RMSE=16.053  MAE=5.122  R²=0.824
```


Predicted vs Actual — SVR (best_estimator_)
Predicted Arrival Delay (min)
Actual Arrival Delay (min)
0
50
100
150
200
250
300
0
200
400
600
800
1000
1200
1400
1600


Residuals vs Fitted — SVR (best_estimator_)
Residual (y - ŷ)
Fitted (Predicted)
1500
1250
1000
750
500
250
0
−250
0
50
100
150
200
250
300



5.3.1 Deep Learning Regression (Keras MLP)

[36]:
```
preA_mlp = clone(preA)
Xtr = preA_mlp.fit_transform(XA_train)
Xte = preA_mlp.transform(XA_test)

# Two hidden layers (128, 64) ~ small "deep" MLP; early stopping on a held-out␣
 ↪val split
mlp_reg = MLPRegressor(
    hidden_layer_sizes=(128, 64),
    activation="relu",
    solver="adam",
    alpha=1e-4,               # L2
    learning_rate_init=1e-3,
    batch_size=128,
    max_iter=500,
    early_stopping=True,
    validation_fraction=0.15,
    n_iter_no_change=10,
    random_state=42,
```

```
    verbose=False
)

mlp_reg.fit(Xtr, np.asarray(yA_train, dtype=float))
y_pred_dl = mlp_reg.predict(Xte)

rmse_dl = root_mean_squared_error(yA_test, y_pred_dl)
mae_dl  = mean_absolute_error(yA_test, y_pred_dl)
r2_dl   = r2_score(yA_test, y_pred_dl)
print(f"[sklearn MLP Regression] RMSE={rmse_dl:.3f}  MAE={mae_dl:.3f} ␣
 ↪R²={r2_dl:.3f}")
```

```
[sklearn MLP Regression] RMSE=10.831  MAE=5.503  R²=0.920
Saved: mlp_sklearn_arrival.joblib, preprocessor_sklearn_mlp.joblib
```

[37]:
```
rows = []

def _metrics(y_true, y_pred):
    return {
        "RMSE": root_mean_squared_error(y_true, y_pred),
        "MAE":  mean_absolute_error(y_true, y_pred),
        "R²":   r2_score(y_true, y_pred)
    }

# Linear
if "linA" in globals():
    y_lin = linA.predict(XA_test)
    m = _metrics(yA_test, y_lin)
    rows.append({"Model": "Linear Regression", **m})

# Ridge
if "ridgeA" in globals():
    y_rdg = ridgeA.predict(XA_test)
    m = _metrics(yA_test, y_rdg)
    rows.append({"Model": "Ridge ( =3.0)", **m})

# SVR (best grid estimator)
if "grid" in globals():
    try:
        y_svr = grid.best_estimator_.predict(XA_test)
        m = _metrics(yA_test, y_svr)
        rows.append({"Model": "SVR (RBF, best grid)", **m})
    except Exception:
        pass

# MLPRegressor (needs transformed X via preA_mlp)
if "mlp_reg" in globals() and "preA_mlp" in globals():
```

```
    Xte_mlp = preA_mlp.transform(XA_test)
    y_mlp = mlp_reg.predict(Xte_mlp)
    m = _metrics(yA_test, y_mlp)
    rows.append({"Model": "MLPRegressor (128,64)", **m})

cmp_df = pd.DataFrame(rows).sort_values("RMSE")
print(cmp_df.to_string(index=False, float_format=lambda x: f"{x:.3f}"))
```

```
                Model   RMSE   MAE    R²
MLPRegressor (128,64) 10.831 5.503 0.920
        Ridge ( =3.0) 10.973 5.367 0.918
    Linear Regression 10.973 5.367 0.918
 SVR (RBF, best grid) 16.053 5.122 0.824
```

5.4 Act

The best-performing model by RMSE was the MLPRegressor (128,64) with RMSE 10.83 minutes, MAE 5.50 minutes, and R^2 0.92. For planning, treat ±5.5 minutes as the typical error band around predictions. We recommend flagging flights with predicted arrival delay > 20 minutes for proactive staffing and customer messaging, and re-evaluating the model monthly to monitor drift (e.g., seasonal shifts or schedule changes).

[ ]: